HDOC 16-1

NAME: DARSHIL RAJ DATE: 15/10/26
SAP ID: 590040196

Program to Check if a Number is a Palindrome

This document provides a clean, well-commented implementation in C that checks whether an integer is a palindrome. A palindromic number reads the same forwards and backwards (e.g., 121 or 1221). The program reverses the number using modulo and division operators inside a while loop and then compares the reversed value with the original.

1. C Implementation

Step 1: Understand the Logic

A number is a palindrome if it is equal to its reverse. To find the reverse, repeatedly extract the last digit using (num % 10), append it using reversed = reversed * 10 + digit, and remove the last digit using num /= 10. Finally, compare reversed with the saved original number.

- **Initialization:** reversed = 0.
- **Condition:** num != 0, so the loop continues while digits remain.
- **Update:** num /= 10 after each extraction.
- **Decision:** if originalNum == reversed, print palindrome; otherwise, not a palindrome.

Step 2: Define Variables and Data Types

Use int for the input number (num), a saved copy (originalNum), the extracted digit (remainder) and the reversed result (reversed). The copy is needed because num is reduced to 0 inside the loop.

Step 3: Take User Input

Prompt the user using printf("Enter an integer: "); and read using scanf("%d", &num);.

Step 4: Reverse and Compare

1. Store the input in originalNum.
2. Extract remainder = num % 10.
3. Update reversed = reversed * 10 + remainder and num /= 10 until num is 0.
4. Use an if-else statement to compare originalNum with reversed and print the result.

Complete C Code

```
/*
 * Q27 (User Inputs, Operations & Output)
 * Write a program to check if a number is a palindrome.
 */
#include <stdio.h>

int main() {
    int num, originalNum, remainder, reversed = 0;

    // Step 1: Read the value of num
    printf("Enter an integer: ");
    scanf("%d", &num);
    originalNum = num;              // Save original for comparison

    // Step 2: Reverse the number using a while loop
    while (num != 0) {
        remainder = num % 10;
        reversed = reversed * 10 + remainder;
        num /= 10;
    }

    // Step 3: Compare reversed number with the original
    if (originalNum == reversed)
        printf("%d is a palindrome.\n", originalNum);
    else
        printf("%d is not a palindrome.\n", originalNum);

    return 0;
}
```

Program Output (Sample Runs)

```
Run 1:  $ gcc hdoc16_1.c -o hdoc16_1
        $ ./hdoc16_1
        Enter an integer: 121
        121 is a palindrome.

Run 2:  $ ./hdoc16_1
        Enter an integer: 12321
        12321 is a palindrome.

Run 3:  $ ./hdoc16_1
        Enter an integer: 123
        123 is not a palindrome.

Run 4:  $ ./hdoc16_1
        Enter an integer: 10
        10 is not a palindrome.

Run 5:  $ ./hdoc16_1
        Enter an integer: 7
        7 is a palindrome.
```

Dry Run with Test Cases

Sample Test Case 1

Input: num = 121 (originalNum = 121)
Calculation:
Start: reversed = 0, num = 121.
Pass 1: 121 != 0 is TRUE. remainder = 1, reversed = 1, num = 12.
Pass 2: 12 != 0 is TRUE. remainder = 2, reversed = 12, num = 1.
Pass 3: 1 != 0 is TRUE. remainder = 1, reversed = 121, num = 0.
Check: 0 != 0 is FALSE, loop terminates.
Compare: originalNum (121) == reversed (121) is TRUE.
Output: 121 is a palindrome.

Sample Test Case 2

Input: num = 123 (originalNum = 123)
Calculation:
Pass 1: remainder = 3, reversed = 3, num = 12.
Pass 2: remainder = 2, reversed = 32, num = 1.
Pass 3: remainder = 1, reversed = 321, num = 0.
Compare: originalNum (123) == reversed (321) is FALSE.
Output: 123 is not a palindrome.

Sample Test Case 3

Input: num = 7
Calculation: Pass 1: remainder = 7, reversed = 7, num = 0. Compare: 7 == 7 is TRUE.
Output: 7 is a palindrome.